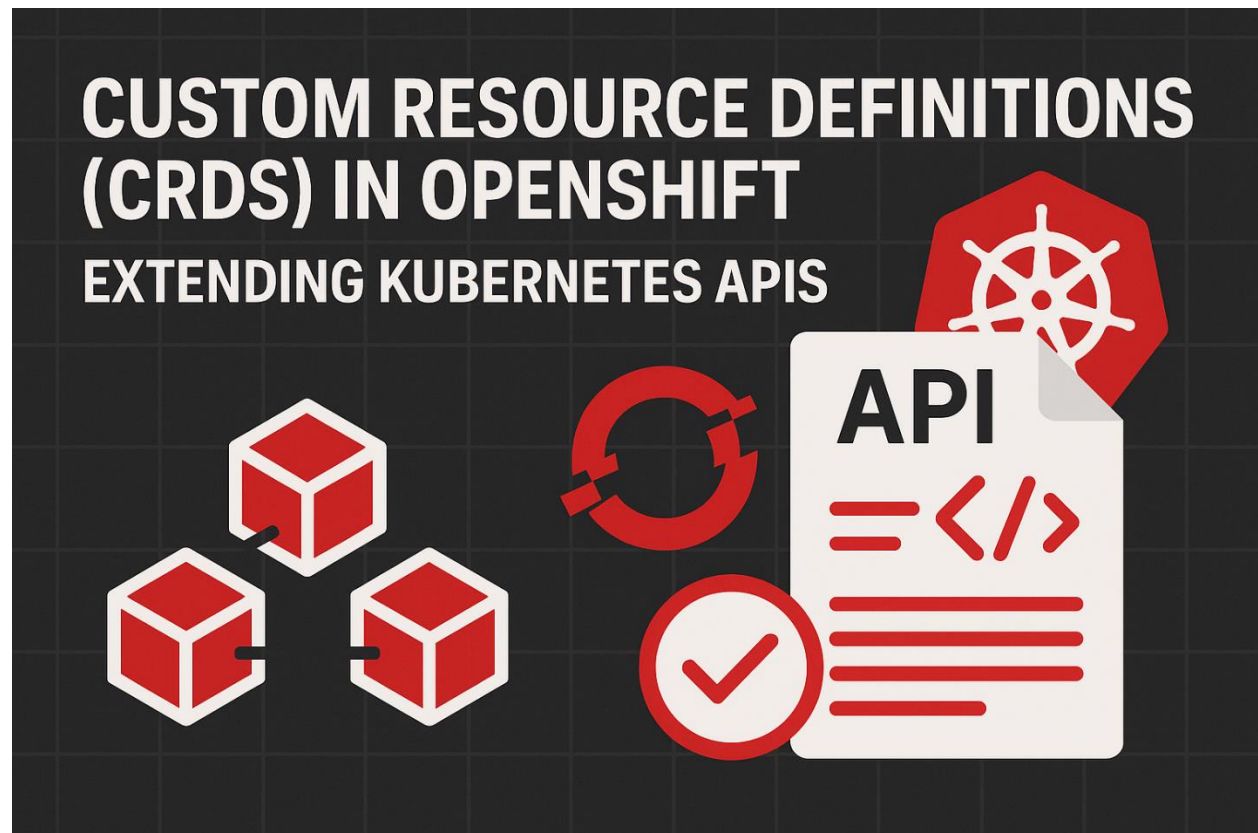


For more details, please visit <https://assistedlabs.com>

Custom Resource Definitions (CRDs) in OpenShift – Extending Kubernetes APIs



Introduction

In Kubernetes and OpenShift, the API server provides a set of built-in objects, such as Pods, Services, and Deployments. However, every application has unique requirements, and built-in resources may not always be enough.

This is where **Custom Resource Definitions (CRDs)** come in.

A **CRD** allows you to define your own custom resources, just like native Kubernetes objects, and manage them using standard Kubernetes tools (kubectl or oc).

Key Benefits of CRDs in OpenShift:

- Extend the Kubernetes API without modifying its core.

To get the OpenShift Hands-on Experience, kindly visit: <https://assistedlabs.com>

For more details, please visit <https://assistedlabs.com>

- Provide declarative APIs for custom applications.
- Integrate with Operators for automation.
- Allow developers to work with domain-specific objects.

Understanding CRDs

A **Custom Resource** is a new type of resource you define in Kubernetes.

A **Custom Resource Definition** is a YAML manifest that tells Kubernetes/OpenShift how to create and handle that resource.

Once a CRD is applied:

- The API server starts recognizing the new resource type.
- Users can create instances of the new resource using YAML files.
- Resources can be viewed, listed, and managed like native objects.

Example Use Cases:

- A database team may define a Database resource to manage MySQL or PostgreSQL clusters.
- A DevOps team may define a BackupPolicy resource for automated backups.
- A machine learning team may define an MLModel resource for model deployments.

Structure of a CRD

A basic CRD YAML contains:

1. **apiVersion** – Usually `apiextensions.k8s.io/v1`
2. **kind** – `CustomResourceDefinition`
3. **metadata** – Name of the CRD
4. **spec** – Defines group, names, scope, versions, and schema.

Example CRD YAML:

To get the OpenShift Hands-on Experience, kindly visit: <https://assistedlabs.com>

ASSISTED LABS

For more details, please visit <https://assistedlabs.com>

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: databases.mycompany.com
spec:
  group: mycompany.com
  versions:
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                engine:
                  type: string
                version:
                  type: string
  scope: Namespaced
  names:
    plural: databases
    singular: database
    kind: Database
    shortNames:
      - db
```

Explanation of Key Fields:

- **group** – API group for the CRD (mycompany.com in this example).
- **versions** – API versions supported for the resource.
- **scope** – Can be Namespaced or Cluster.
- **names** – Defines how the resource is referred to in commands.

Creating and Using CRDs in OpenShift

Step 1 – Create the CRD

```
oc apply -f database-crd.yaml
```

To get the OpenShift Hands-on Experience, kindly visit: <https://assistedlabs.com>

ASSISTED LABS

For more details, please visit <https://assistedlabs.com>

Step 2 – Verify CRD creation

```
oc get crd
```

Step 3 – Create a Custom Resource (CR) instance

```
apiVersion: mycompany.com/v1
kind: Database
metadata:
  name: mydb
spec:
  engine: mysql
  version: "8.0"
bash
CopyEdit
oc apply -f mydb.yaml
```

Step 4 – View the resource

```
oc get databases
oc describe database mydb
```

CRDs vs Operators

In OpenShift, CRDs are often used together with **Operators**.

- **CRD** defines the API schema and object type.
- **Operator** contains the logic to manage those custom resources automatically.

Example:

- The Database CRD defines how a database resource looks.
 - The Database Operator watches for Database resources and performs tasks like provisioning, scaling, or backups.
-

Best Practices for CRDs

- Use clear naming conventions to avoid conflicts.
- Version your CRDs (v1alpha1, v1beta1, v1) to manage changes.

To get the OpenShift Hands-on Experience, kindly visit: <https://assistedlabs.com>

For more details, please visit <https://assistedlabs.com>

- **Validate with schemas** to prevent invalid data.
 - **Document your CRDs** so developers know how to use them.
 - **Integrate with RBAC** to control access to custom resources.
-

Advantages of Using CRDs in OpenShift

- No need to modify Kubernetes core code.
 - Works with Kubernetes-native tools (oc, kubectl).
 - Can be used with GitOps workflows (e.g., ArgoCD).
 - Flexible for various industries (finance, healthcare, AI/ML, etc.).
-

Limitations of CRDs

- CRDs alone do not contain automation logic — you need Operators or controllers for that.
 - Schema changes between versions require careful migration.
 - Poorly designed CRDs can lead to maintenance complexity.
-

Real-World Examples

- **OpenShift Pipelines:** Uses CRDs to define Pipeline, Task, and PipelineRun.
 - **Prometheus Operator:** Uses CRDs like Prometheus and Alertmanager.
 - **Service Mesh:** Istio CRDs define custom networking resources.
-

Conclusion

Custom Resource Definitions (CRDs) are a powerful way to extend Kubernetes APIs in OpenShift. They enable teams to create domain-specific objects that behave like native Kubernetes resources.

When combined with Operators, CRDs become the foundation for automation, self-healing applications, and infrastructure-as-code practices.

To get the OpenShift Hands-on Experience, kindly visit: <https://assistedlabs.com>